

INFORME FINAL DEL PROYECTO — MUELLE DIGITAL

Contribución por rol del equipo de desarrollo

Proyecto: Muelle Digital — Marketplace de experiencias ecoturísticas fluviales (Barrancabermeja) **Materia:** Ingeniería de Software **Equipo:**

- Esteban Merlano — Líder del Proyecto · Backend · Despliegue
- Keyner Trujillo — Frontend
- Andres Rangel — Base de Datos
- Jhon Escobar — Aseguramiento de Calidad (QA)

Aplicación en producción:

- App: <https://muelle-digital.vercel.app>
- API: <https://muelle-digital-api.vercel.app>
- Documentación API (Swagger): <https://muelle-digital-api.vercel.app/api/docs>

1. Resumen del proyecto

Muelle Digital es una plataforma web tipo marketplace que centraliza la oferta de experiencias ecoturísticas fluviales en Barrancabermeja (avistamiento de manatíes, pesca artesanal, paseos en canoa, aviturismo). Conecta a **turistas** con **guías y pescadores locales verificados**, permitiéndoles buscar, reservar, **pagar en línea** y calificar experiencias, con el **punto de encuentro geolocalizado en un mapa**.

El proyecto resuelve la informalidad del ecoturismo fluvial: falta de un canal centralizado de reservas, precios poco claros, ausencia de mecanismos de confianza y de pagos digitales. El producto se desarrolló como MVP completo, se probó de extremo a extremo y se **desplegó en producción**.

Stack tecnológico general:

Capa	Tecnologías
Frontend	React 18, Vite, TailwindCSS, React Router, Axios, MapLibre GL, lucide-react
Backend	Node.js, Express (API REST), JWT, bcrypt, Stripe

Base de datos	PostgreSQL en NeonDB (driver <code>pg</code>)
Documentación API	OpenAPI 3 + Swagger UI
Despliegue	Vercel (frontend + backend serverless) + Neon
Gestión y versiones	Git/GitHub (GitFlow), ClickUp

2. Líder del Proyecto — Esteban Merlano

Qué hizo: lideró la planeación, organización y entrega del proyecto, y coordinó el trabajo de los cuatro roles.

- **Definición y análisis:** redactó las **historias de usuario** (13 en total) con sus criterios de aceptación, definió los **roles** del equipo y los actores del sistema (turista, guía, administrador).
- **Planeación:** elaboró el **cronograma de 8 semanas** (definición → arquitectura → 4 sprints → QA → entrega), con hitos y dependencias, **análisis de riesgos** con su plan de mitigación y un **presupuesto estimado**.
- **Gestión ágil:** organizó el tablero de **ClickUp** con los entregables por semana, las ceremonias (dailies, cierres de sprint, retrospectivas) y las métricas de seguimiento.
- **Control de versiones:** configuró el repositorio **Git** con el flujo **GitFlow** y las ramas por rol (produccion, develop, backend, frontend, base-de-datos, qa).
- **Decisiones técnicas clave:** eligió el stack (Node + React + PostgreSQL), justificó **no usar ORM** (SQL parametrizado, sin sobreingeniería) y **Stripe Checkout** por seguridad.
- **Integración:** unió el trabajo de todos los roles y aseguró que el sistema funcionara como un todo.

Resultado: proyecto entregado, integrado y **publicado en producción**.

3. Backend — Esteban Merlano

Qué hizo: construyó toda la **API REST** y la lógica del negocio, con seguridad y pagos.

- **Tecnologías:** Node.js + Express (módulos ESM), pg (PostgreSQL), jsonwebtoken, bcryptjs, stripe, cors, dotenv.
- **Arquitectura por capas:** config/ (conexión a BD y Stripe), middleware/ (autenticación y roles), controllers/ (lógica) y routes/ (endpoints). Código limpio, modular y mantenible.
- **Autenticación y seguridad:** registro/login con contraseñas **encriptadas (bcrypt)** y **tokens JWT**; middleware de control por rol; el rol "guía" solo lo otorga el administrador (no se puede auto-asignar).

- **Funcionalidades:** CRUD de experiencias, **reservas con control de cupos**, sistema de reseñas, gestión de **solicitudes de guía** y endpoints de **administración** (usuarios, métricas).
- **Pagos con Stripe (modo test):** al crear una experiencia se genera automáticamente el **Producto y Precio en Stripe**; al reservar se crea una **Checkout Session**; la confirmación es **idempotente** mediante webhook y verificación de sesión. Preparado para comisiones por guía (modelo marketplace).
- **Documentación de la API:**Swagger / **OpenAPI 3** disponible en /api/docs.

Resultado: API REST completa, segura y documentada, desplegada como función serverless.

4. Frontend — Keyner Trujillo

Qué hizo: desarrolló toda la **interfaz de usuario** y su conexión con la API.

- **Tecnologías:** React 18 + Vite, TailwindCSS, React Router, Axios, MapLibre GL, lucide-react.
- **Pantallas:** landing con grid de experiencias, **login/registro** con diseño de pantalla partida, **catálogo maestro-detalle** (lista + detalle), detalle de experiencia con **mapa y reseñas en tiempo real**, **panel del guía**, **panel del administrador**, "mis reservas" y formulario de verificación de guía.
- **Integración con el backend:** consumo de la API REST con **Axios**, con interceptores que adjuntan el token JWT y normalizan los errores.
- **Mapas:** componentes para que el guía **marque el punto de encuentro** (clic/arrastrar pin) y para que el turista lo **visualice** (MapLibre + OpenFreeMap, gratis y sin API key).
- **Identidad visual (UX/UI):**tema **morado de marca** basado en el logo, **modo claro/oscuro** persistente, logo con contorno blanco, diseño **responsive** y accesibilidad básica.
- **Buenas prácticas:** componentes reutilizables, **rutas protegidas** por rol, manejo de estados de carga/vacío/error.

Resultado: aplicación web moderna, responsive y consumiendo la API en producción.

5. Base de Datos — Andres Rangel

Qué hizo: diseñó y modeló la **base de datos relacional**, fuente de verdad del sistema.

- **Tecnología:**PostgreSQL en **NeonDB** (serverless), con acceso por consultas **parametrizadas** (previene inyección SQL).
- **Modelo de datos:** 6 entidades — usuarios, experiencias, reservas, ratings, pagos y solicitudes_guía — con su **diagrama Entidad-Relación** y **diccionario de datos**.
- **Relaciones:** un guía publica muchas experiencias; un turista hace muchas reservas; cada reserva genera pagos; reseñas únicas por usuario/experiencia. Con **integridad referencial** (claves foráneas + ON DELETE CASCADE).
- **Normalización:** modelo en **Tercera Forma Normal (3FN)**, sin redundancias ni dependencias transitivas.

- **Seguridad e integridad:** contraseñas como hash **bcrypt**, conexión **SSL** obligatoria, restricciones **CHECK** (precio ≥ 0 , puntuación 1–5, estados válidos) y **UNIQUE** (email).
- **Rendimiento:** índices en las claves foráneas y campos de búsqueda más usados.
- **Entregables:** scripts `schema.sql`, `seed.sql` (datos de ejemplo) y un migrador (`migrate.js`).

Resultado: base de datos normalizada, segura y conectada en la nube (Neon), funcionando en producción.

6. Aseguramiento de Calidad (QA) — Jhon Escobar

Qué hizo: garantizó que el software fuera **confiable y seguro**, especialmente por manejar datos personales y pagos.

- **Plan de pruebas:** estrategia de testing con **22 casos** (15 funcionales + 7 de seguridad) y matriz de cobertura.
- **Pruebas funcionales:** registro, login, búsqueda con filtros, reservas con y sin cupos, pagos con Stripe, reseñas y verificación de guías.
- **Pruebas de seguridad:** contraseñas encriptadas, acceso bloqueado sin token (401), acceso por rol (403), imposibilidad de pagar reservas ajenas, protección contra inyección SQL y verificación de que las claves no se suben al repositorio.
- **Reporte de bugs:** documentó y dio seguimiento a **3 defectos** (todos cerrados), por ejemplo el riesgo de auto-asignarse rol de guía, que se corrigió.
- **Revisión de código (Code Review):** uso de **Pull Requests** de Git para que un compañero revisara el código antes de integrarlo.
- **Métricas de calidad:** 100% de casos exitosos tras correcciones; 13/13 historias de usuario cubiertas.

Resultado: plataforma validada funcional y de forma segura, lista para usuarios reales.

7. Despliegue e integración (Líder)

El líder desplegó la solución completa en la nube, **gratis**:

- **Base de datos:** NeonDB (PostgreSQL serverless).
- **Backend:** Vercel (función serverless) → <https://muelle-digital-api.vercel.app>.
- **Frontend:** Vercel (Vite) → <https://muelle-digital.vercel.app>.
- **Configuración segura:** las claves (base de datos, JWT, Stripe) **no están en el código**; se cargaron como **variables de entorno cifradas** en Vercel. El CORS del backend está restringido al dominio del frontend.

Se verificó en producción: estado de salud del backend, consultas a la base de datos, CORS y carga del frontend — **todo correcto**.

8. Estado final

Aspecto	Estado
Historias de usuario (13)	✅ Implementadas
Backend (API + seguridad + pagos)	✅ Completo y documentado (Swagger)
Frontend (todas las pantallas)	✅ Completo y responsive
Base de datos (6 tablas, 3FN)	✅ Modelada y conectada
QA (22 casos + seguridad)	✅ Probado, bugs cerrados
Despliegue en producción	✅ En vivo en Vercel + Neon

Conclusión: el equipo entregó un producto fullstack funcional, seguro y desplegado, donde cada rol cumplió sus responsabilidades y se integró con los demás para lograr el objetivo común.